Desenvolvimento de IA para Análise Preditiva [T1 e T2]

Situação de Aprendizagem (Projeto Avaliativo) - Módulo 2 - Semana 13

SUMÁRIO

1 CONTEXTUALIZAÇÃO 1
2 DESAFIO 2
3 RESULTADOS ESPERADOS (ENTREGA) 2
4 REQUISITOS DA APLICAÇÃO 2
Fase 2: Estrutura Modular da API FastAPI 3
Fase 3: Ciclo de Vida e Inicialização Otimizada (Lifespan) 3
Fase 4: Modelagem e Validação Estrita de Dados (Pydantic) 3
Fase 5: Implementação de Endpoints e Tratamento de Erros (FastAPI & HTTP) 3
5 ROTEIRO DA APLICAÇÃO 4
5.1 FORMATO DO SISTEMA 4
5.2 DOCUMENTAÇÃO NO README.MD 4
5.3 USO DO GITHUB 5
5.4 GRAVAÇÃO DE VÍDEO 5
6 CRITÉRIOS DE AVALIAÇÃO 6
7 PLANO DE PROJETO 7
8 SOBRE O USO DE INTELIGÊNCIA ARTIFICIAL 7

1 CONTEXTUALIZAÇÃO

Na Ciência de Dados e Engenharia de Machine Learning moderna, a criação de modelos preditivos com alta precisão representa uma parte importante do ciclo de desenvolvimento. Mas, o verdadeiro valor de negócio é alcançado quando esses algoritmos saem do ambiente experimental (Jupyter Notebooks) e são colocados em produção na forma de **serviços escaláveis, integráveis e de baixa latência**.

No mercado de tecnologia, desenvolvedores e cientistas de dados precisam atuar de forma convergente com as práticas de MLOps: empacotar pipelines treinados, gerenciar ciclos de vida de aplicações web assíncronas, assegurar validação e integridade de tipos em tempo de execução e construir interfaces padronizadas (APIs RESTful). O **FastAPI**, suportado por servidores ASGI como o Uvicorn e pela validação rigorosa do Pydantic, consolidou-se como um dos padrões da indústria para disponibilização de microsserviços preditivos.

Neste Projeto Avaliativo Final, você atuará como Engenheiro de Machine Learning/Desenvolvedor Back-end, sendo responsável por disponibilizar um modelo preditivo treinado como uma API REST pronta para consumo por aplicações externas.

2 DESAFIO

Você deverá selecionar **um modelo de Machine Learning**, optando por:

- **Opção A:** Reaproveitar e/ou aprimorar um dos modelos desenvolvidos nos 3 últimos projetos do curso (ex.: Previsão de Risco/Score de Crédito, Classificação de Imagens MNIST/Visão Computacional, Análise de Churn/Regressão Logística ou outros, dependendo da turma);

OU

- **Opção B:** Desenvolver e treinar um pipeline preditivo próprio do zero a partir de uma base de dados pública justificada (Kaggle, OpenML, UCI Repository ou outra).

A partir do modelo treinado e serializado, você deverá construir, modularizar e documentar uma **API Web completa com FastAPI**, implementando endpoints padronizados de monitoramento de saúde (/health) e de inferência sob demanda (/predict ou /v1/predict), com esquemas de entrada e saída validados via Pydantic e ciclo de vida otimizado.

3 RESULTADOS ESPERADOS (ENTREGA)

O código deverá ser inserido e versionado no **GitHub em modo público**, o vídeo deverá ser inserido no **Google Drive em modo leitor para qualquer pessoa** com o link. Ambos os **links** deverão ser disponibilizados na tarefa **Módulo 2 - Projeto Avaliativo**, presente **na Semana 13 do módulo 02** do AVA até o dia **03/11/2026 às 22h.**

Não serão aceitos projetos submetidos após a data limite da atividade, e, ou alterados depois de entregues.

Importante:

1. Será considerado como data final de entrega a última atualização no repositório do projeto no GitHub. Lembre-se de não modificar o código até receber sua nota.
2. **Não esqueça de submeter os links no AVA (Vídeo e GitHub)**. Os links não submetidos terão penalidade na nota.

4 REQUISITOS DA APLICAÇÃO

O projeto deve, obrigatoriamente, cumprir as seguintes etapas técnicas:

Fase 1: Preparação e Serialização do Modelo (Pipeline Offline)

- Treinar (ou refinar) o modelo preditivo escolhido, garantindo a divisão entre dados de treino, validação e teste e os devidos tratamentos de pré-processamento (imputação de nulos, codificação categórica e escalonamento).
- **Serialização do Artefato:** Salvar o pipeline ou modelo treinado em um arquivo binário dedicado (ex.: `.joblib` ou `.pkl`) dentro de uma pasta apropriada do projeto (ex.: `models/`) ou usar um banco de dados sqlite, como explorado em aula ou ainda uma ferramenta mais apropriada e robusta como o MLflow.
- **Relatório de Métricas:** Apresentar em documentação ou script auxiliar as métricas alcançadas pelo modelo selecionado (Acurácia, F1-Score, ROC-AUC, MSE/R^2 dependendo do tipo de problema).

Fase 2: Estrutura Modular da API FastAPI

- A aplicação **não** pode ser um arquivo único com todo o código misturado. Deve seguir uma arquitetura em camadas:
  - `schemas/` ou `schemas.py`: Definição de modelos Pydantic (`BaseModel`, `Field`) para validação estrita do JSON recebido no corpo da requisição e do JSON retornado.
  - `services/` ou `services.py`: Módulo responsável por carregar o modelo serializado e executar a inferência matemática, desacoplando regras de negócio do protocolo HTTP.
  - `routers/` ou `routers.py`: Módulos de rota usando `APIRouter` agrupando os endpoints de negócio.
  - `main.py`: Ponto de entrada da aplicação, inclusão dos roteadores e gerenciamento do ciclo de vida.

Fase 3: Ciclo de Vida e Inicialização Otimizada (Lifespan)

- O arquivo serializado do modelo **deve ser carregado na memória RAM uma única vez na inicialização da aplicação** (utilizando `@asynccontextmanager` com `lifespan` do FastAPI).
- **Proibição Técnica:** É terminantemente proibido ler o disco ou recarregar o arquivo `.joblib`/`.pkl` dentro da função da rota de predição a cada requisição HTTP recebida.

Fase 4: Modelagem e Validação Estrita de Dados (Pydantic)

- **Esquemas de Entrada e Saída:** Criar classes herdadas de `BaseModel` em módulo separado (`schemas.py`), definindo explicitamente o contrato de dados que a API aceita e o contrato que ela devolve.
- **Uso de Tipagem e Restrições (`Field`):** Configurar tipos estáticos adequados e regras mínimas de negócio nos atributos (ex.: faixas de valores com `ge`, `le`, `gt`, limites de tamanho de texto e descrições).
- **Prevenção de Dados Inválidos:** Garantir que o envio de tipos incorretos (ex.: texto em campos numéricos ou valores fora dos limites) seja interceptado automaticamente

pelo Pydantic, gerando respostas com status HTTP `422 Unprocessable Entity` antes de repassar os dados para o modelo preditivo.

Fase 5: Implementação de Endpoints e Tratamento de Erros (FastAPI & HTTP)

- **Endpoint de Monitoramento (`GET /health`):** Rota de checagem de integridade operacional que deve responder com status HTTP `200 OK`, informando se o serviço está ativo e se o modelo de Machine Learning reside corretamente na memória RAM.
- **Endpoint de Inferência (`POST /predict` ou `/v1/predict`):** Rota que consome o esquema de entrada validado, invoca a camada de serviço/inferência e retorna a predição no formato do esquema de saída configurado com status HTTP `200 OK`.
- **Semântica e Tratamento de Exceções HTTP:** Mapear e retornar status codes apropriados (`200 OK`, `400 Bad Request`, `422 Unprocessable Entity` ou `500 Internal Server Error`), evitando que a API caia (*crash*) ou retorne erros sem tratamento ao usuário.

5 ROTEIRO DA APLICAÇÃO

Abaixo segue o detalhamento de como a solução deve ser estruturada, organizada e entregue no repositório do projeto.

5.1 FORMATO DO SISTEMA

A aplicação é um Pipeline de Ciência de Dados ponta a ponta, incluindo o deploy, construído em Python. O repositório do projeto deve seguir a estrutura e organização de entrega:

- **Entrega (.ipynb ou py):** aceito como notebook (.ipynb) ou Python puro (.py).
- **Código Limpo e Explicativo:** organizado com cabeçalhos em Markdown estruturando todas as Fases e Desafios, justificando as tomadas de decisões técnicas (no pipeline de machine learning).
- **Reprodutibilidade (requirements.txt):** O repositório deve conter um arquivo de texto listando todas as dependências de bibliotecas Python utilizadas (como pandas, scikit-learn, seaborn, tensorflow, keras, fastapi e suas respectivas versões), permitindo que qualquer outro cientista de dados consiga recriar o ambiente e rodar o pipeline/api sem erros.
- **Caminhos Relativos de Dados:** Se os dados forem baixados e lidos localmente no repositório (em vez de carregados por URL direta), eles devem ser organizados em uma pasta de dados específica (ex: /data). É proibido o uso de caminhos absolutos que apontem para diretórios locais da sua máquina (ex: C:/Usuarios/SeuNome/Documents/...), pois isso impede a execução do código pelo avaliador. Não esqueçam de organizar as próprias imagens digitalizadas utilizadas no Desafio C.

5.2 DOCUMENTAÇÃO NO README.MD

O arquivo README.md é a vitrine do seu projeto/portfólio profissional. Ele deve conter obrigatoriamente:

1. **Nome e Resumo do Projeto:** Qual problema de negócio o modelo preditivo resolve.
2. **Origem do Modelo:** Indicar qual dos 3 projetos anteriores foi aproveitado ou descrever a base utilizada para o projeto próprio.
3. **Tecnologias Utilizadas:** Listar bibliotecas e versões (FastAPI, Pydantic, Scikit-Learn, Uvicorn, etc.).
4. **Instruções de Instalação e Execução:** Comandos para criação de ambiente virtual (venv), instalação do requirements.txt e inicialização do servidor com Uvicorn.
5. **Documentação da API:** Exemplos de payloads JSON para chamada do endpoint /predict e link da documentação Swagger (/docs).
6. **Links das Evidências:** Link do vídeo no Google Drive.

5.3 USO DO GITHUB

Utilizar um repositório GitHub para controle de versionamento do projeto. Siga as diretrizes de boas práticas de uso do Git abaixo:

- Seguir *commits* com descrição sucinta e direta e, de preferência, com poucos arquivos alterados por *commit*.
    - Prefira mensagens como “implementa X” ou “corrige Y”, em vez de “X foi implementado” ou “Y foi corrigido”.
- Crie uma *branch* “develop” para concentrar os merges enquanto estiver desenvolvendo as funcionalidades.
- Utilizar *feature branches* a partir da *develop* para cada tarefa.
    - Não excluir a branch após o pull request concluído.
- O código final do projeto deve estar todo “mergeado” no *branch* “main”.

5.4 GRAVAÇÃO DE VÍDEO

Além do desenvolvimento deste sistema você deverá gravar um vídeo, com tempo **máximo de 10 minutos**, abordando os seguintes questionamentos:

- Qual o objetivo do sistema? E demonstração de funcionamento.
- O que deve ser realizado para executar o sistema?
- Como você organizou as tarefas antes de começar a desenvolver?
- Quais as *branches* você criou e quais os objetivos para cada uma?
- Você acha que faltou algo no seu código que você poderia melhorar?
- Argumentação clara, baseada nos conteúdos abordados, de suas escolhas realizadas nesse projeto.

Você poderá gravar na vertical ou na horizontal. É importante que apareça seu rosto e esteja em um local com boa iluminação. Para realizar a entrega do vídeo, coloque em uma pasta

do Google Drive em modo leitor para qualquer pessoa com o link, e compartilhe o mesmo na submissão do projeto no AVA. Uma dica interessante é você inserir o vídeo no readme.md do seu projeto no repositório do GitHub.

Note: Não devem ser utilizadas qualquer ferramenta de criação de vídeo com IA nessa etapa.

6 CRITÉRIOS DE AVALIAÇÃO

A tabela abaixo apresenta os critérios que serão avaliados durante a correção do projeto. O mesmo possui variação de nota de 0 (zero) a 10 (dez) como nota mínima e máxima, e possui peso de 60% sobre a avaliação do módulo.
Serão desconsiderados e atribuída a nota 0 (zero) os projetos que apresentarem plágio de soluções encontradas na internet ou de outros colegas. Lembre-se: Você está livre para utilizar outras soluções como base, mas não é permitida a cópia.

Apresentação do Projeto (Nota máxima no bloco: 2,0)			
Nº	Critério de Avaliação	0	2,0
1	**Realizou a gravação de um vídeo?**	**NÃO REALIZADO.** Não foi realizada a gravação do vídeo.	**TOTALMENTE.** Gravou o vídeo e abordou todos os tópicos listados no item 5.4

Uso adequado do GitHub e Readme.md (Nota máxima no bloco: 3,0)				
Nº	Critério de Avaliação	0	0,5	1.5
2	**Foi aberto um branch separado para cada etapa, com mensagens do commit claras e adequadas?**	**NÃO REALIZADO.** Os commits foram realizados diretamente na master/main, e não foi usado mensagens claras.	**PARCIALMENTE.** Foram abertas branchs para cada funcionalidade, porém foram realizados commits de mesma funcionalidade em branchs diferentes, ou mensagens inadequadas.	**TOTALMENTE.** Foram abertas branchs para cada funcionalidades, e os commits foram realizados de forma concisa com mensagens adequadas.
3	**Criou uma documentação com readme.md?**	**NÃO REALIZADO.** Não criou a documentação.	**PARCIALMENTE.** Criou uma documentação, porém de forma muito simplificada.	**TOTALMENTE.** Criou uma documentação completa com ao menos todos os tópicos sugeridos no item 5.2

Desenvolvimento da Aplicação (Nota máxima no bloco: 5,0)				
Nº	Critério de Avaliação	0	0.5	1.0

4	**<u>Fase 1:</u> Serialização e Modelo de Machine Learning**	**NÃO REALIZADO.** Não possui modelo treinado ou artefato serializado.	**PARCIALMENTE.** Treinou modelo, mas o arquivo de pesos está corrompido ou mal configurado.	**TOTALMENTE.** Modelo funcional, serializado adequadamente em disco e com métricas documentadas.
5	**<u>Fase 2:</u> Arquitetura Modular do Projeto**	**NÃO REALIZADO.** Código em arquivo único ou desestruturado, sem pacotes Python.	**PARCIALMENTE.** Divisão parcial, misturando regras do modelo diretamente dentro das rotas.	**TOTALMENTE.** Projeto estruturado em camadas (routers, schemas, services, main).
6	**<u>Fase 3:</u> Ciclo de Vida e Carregamento (Lifespan)**	**NÃO REALIZADO.** Não carrega o modelo ou recarrega o arquivo do disco a cada requisição preditiva.	**PARCIALMENTE.** Carregou o modelo fora do lifespan em escopo instável ou redundante.	**TOTALMENTE.** Carregou o artefato no ciclo de vida da aplicação (lifespan/startup), mantendo-o em memória RAM.
7	**<u>Fase 4:</u> Modelagem e Validação Estrita de Dados (Pydantic)**	**NÃO REALIZADO.** Não utilizou Pydantic (uso de dicionários brutos sem tipagem).	**PARCIALMENTE.** Utilizou Pydantic apenas na entrada ou sem validação de regras/campos (Field).	**TOTALMENTE.** Definiu esquemas completos de entrada e saída com restrições (Field), gerando erro 422 de forma consistente para dados inválidos.
8	**<u>Fase 5:</u> Implementação de Endpoints e Tratamento de Erros (FastAPI & HTTP)**	**NÃO REALIZADO.** Endpoints não funcionam, geram erro 500 no Swagger ou não atendem às rotas solicitadas.	**PARCIALMENTE.** Apenas um dos endpoints está funcional, ou as rotas devolvem respostas fora da estrutura esperada.	**TOTALMENTE.** Ambos os endpoints funcionam no /docs, respondendo com os códigos HTTP semânticos corretos (200 OK, 422, 400).

7 PLANO DE PROJETO

Ao concluir esta situação de aprendizagem, o estudante demonstra domínio prático em:

- **Engenharia de Software:** Aplicar a metodologia ágil e controle de versão profissional via Git/GitHub sob o modelo GitFlow.
- **MLOps e Deploy de Modelos:** Persistir e desserializar pipelines preditivos sem vazamento de dados ou gargalos de Entrada/Saída (I/O).
- **Desenvolvimento Web Back-end:** Estruturar microsserviços modernos, assíncronos e orientados a eventos com **FastAPI** e servidores ASGI (Uvicorn).
- **Qualidade e Validação:** Implementar tipagem forte e validação de esquemas de dados em tempo de execução com **Pydantic**.
- **Comunicação Técnica:** Sintetizar e defender tecnicamente suas escolhas de arquitetura e implementação através de documentação técnica e apresentação em vídeo.

8 SOBRE O USO DE INTELIGÊNCIA ARTIFICIAL

É proibida a geração integral e não supervisionada do código por ferramentas geradoras de código. O estudante deve ser o autor do código e compreender cada linha submetida. O uso de ferramentas de IA é permitido exclusivamente como suporte de estudos conceituais, depuração de erros de sintaxe ou esclarecimento de dúvidas teóricas sobre a documentação das bibliotecas. O projeto será arguido com base no domínio técnico demonstrado no vídeo e no repositório.